

Day 78: Matplotlib Fundamentals

1. Day Number

Day 78

2. Topic Name

Basic Data Visualization with Matplotlib

Today you will learn how to turn simple numerical data into charts using Python.

3. Connection

You can now store, inspect, and clean numerical data using tools such as NumPy and Pandas.

Today you will take the next step:

Data → Clean Data → Visual Chart

A chart can make patterns much easier to understand than looking at numbers alone.

4. Important Topics

- Matplotlib
- `figure`
- axes
- line chart
- bar chart
- chart title
- x-axis label
- y-axis label

The common Matplotlib module used for beginner charts is:

```
import matplotlib.pyplot as plt
```

You will often see `plt` used as the short name for `matplotlib.pyplot`.

5. Foundational Notes

What is Matplotlib?

Matplotlib is a Python library used to create charts and graphs.

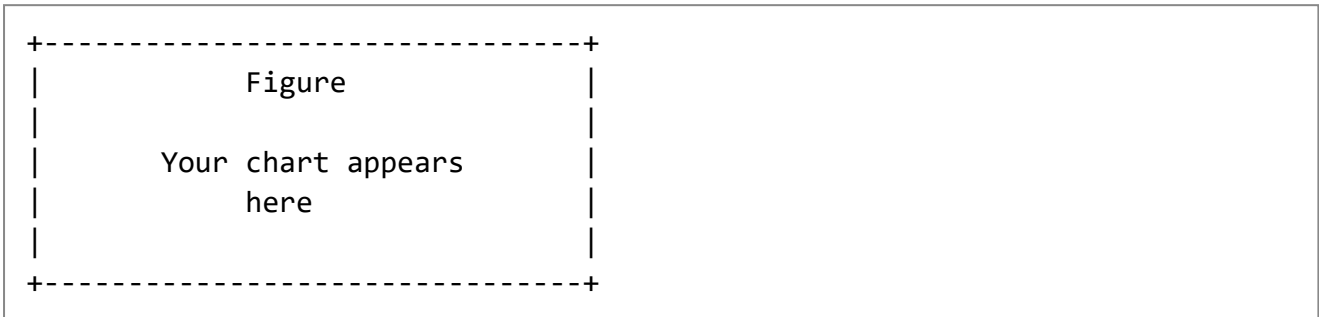
For example, suppose you have monthly sales:

```
January → 100  
February → 140  
March → 120
```

You could read these numbers directly, but a chart may help you notice changes faster.

What is a figure?

Think of a **figure** as the entire drawing area or canvas containing your chart.



What are axes?

The **axes** are the actual area where your data is plotted.

A simple chart normally has:



At this stage, you do not need to work with Matplotlib's more advanced figure-and-axes system directly. Just understand the idea.

Basic Matplotlib flow

A beginner chart often follows this pattern:

```
import matplotlib.pyplot as plt

# prepare data

# create chart

# add title

# add axis labels

# display chart
```

The command commonly used to display the finished chart is:

```
plt.show()
```

6. When Are Line Charts and Bar Charts Useful?

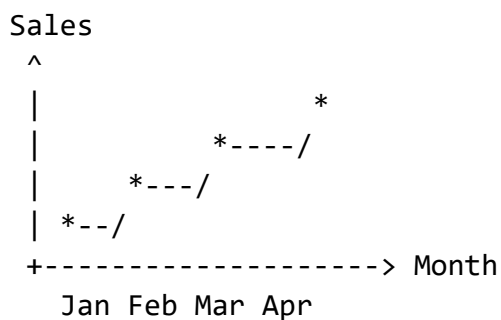
Line chart

A **line chart** is especially useful when values change across an ordered sequence, particularly over time.

Examples:

- monthly sales
- daily temperature
- website visitors each week
- yearly profit

Imagine:



The connecting line makes the trend easier to see.

A line chart is commonly created using:

```
plt.plot(...)
```

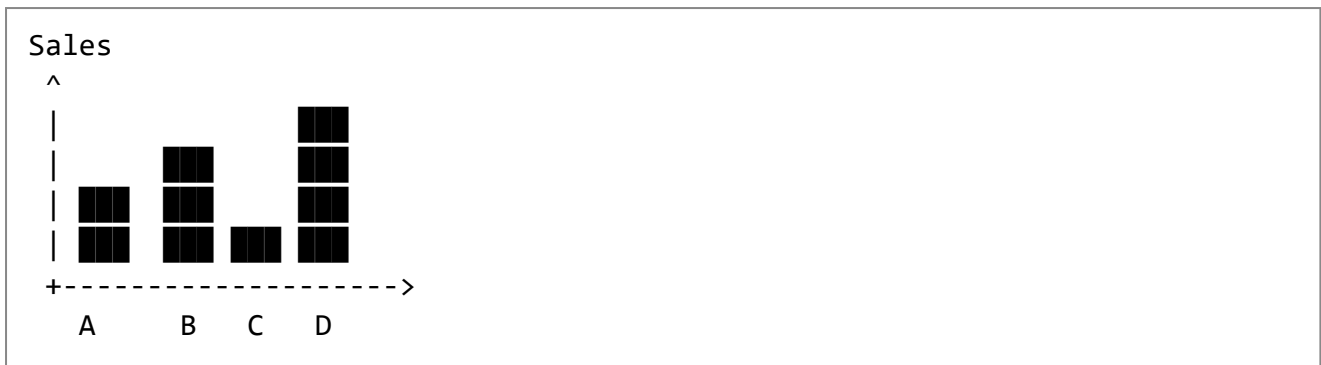
Bar chart

A **bar chart** is useful when comparing separate categories.

Examples:

- sales of different products
- students in different classes
- expenses by category
- orders from different cities

Example:



A bar chart is commonly created using:

```
plt.bar(...)
```

For today's monthly-sales exercise, either a line chart or bar chart would be reasonable.

7. Easy Example

Suppose we have temperatures for three days:

```
days = ["Mon", "Tue", "Wed"]  
temperatures = [28, 30, 29]
```

Conceptually, we could create a line chart using:

```
plt.plot(days, temperatures)
```

Then we might add:

```
plt.title(...)  
plt.xlabel(...)  
plt.ylabel(...)
```

And finally display it with:

```
plt.show()
```

Notice that the first list provides the **x-axis values**, while the second list provides the corresponding **y-axis values**.

So:

```
Mon → 28  
Tue → 30  
Wed → 29
```

The two lists should normally contain matching numbers of values.

8. Problem Statement

Create a very small dataset containing monthly sales.

For example:

Month	Sales
January	120
February	150
March	135
April	180

Your program should:

1. Store the months.
2. Store their sales values.
3. Create a simple line chart or bar chart.
4. Give the chart a meaningful title.
5. Label the x-axis.
6. Label the y-axis.
7. Display the chart.

You may use normal Python lists or a tiny Pandas DataFrame.

Keep the exercise small.

9. Concepts Used

For this problem, you will practice:

- Python lists or a Pandas DataFrame
- numerical data
- categorical/month labels
- importing a library
- Matplotlib
- `plt.plot()` or `plt.bar()`
- `plt.title()`
- `plt.xlabel()`
- `plt.ylabel()`
- `plt.show()`

You are combining your earlier data skills with your first visualization skill.

10. Thought Process

Before writing code, think through the problem like this:

Step 1: What should appear horizontally?

The months:

Jan, Feb, Mar, Apr

These belong on the **x-axis**.

Step 2: What should appear vertically?

The sales numbers:

120, 150, 135, 180

These belong on the **y-axis**.

Step 3: What chart makes sense?

Because the values represent sales changing month by month, a **line chart** is useful for seeing the trend.

A bar chart would also work if your main goal were simply comparing the monthly values.

Step 4: What information should explain the chart?

Add:

Title: Monthly Sales
X-axis: Month
Y-axis: Sales

Step 5: How will the chart appear?

After preparing everything, display it using Matplotlib.

11. Pseudocode

```
START

import the Matplotlib plotting module

create a list containing month names

create a list containing sales values

create a line chart using months and sales

add a chart title

add an x-axis label

add a y-axis label
```

```
display the chart
```

```
END
```

For a bar chart, simply replace the conceptual line-chart step with:

```
create a bar chart using months and sales
```

12. Suggested Solving Approach: Matplotlib

A simple structure for your program could be:

```
import matplotlib.pyplot as plt

# Step 1: prepare months

# Step 2: prepare sales numbers

# Step 3: create the chart

# Step 4: add title

# Step 5: label x-axis

# Step 6: label y-axis

# Step 7: display chart
```

Focus only on these basics today.

You do **not** need to worry about colors, styles, legends, multiple charts, subplots, annotations, or advanced customization yet.

13. Easy Edge Cases

One data point

Suppose you only have:

```
January → 120
```

A bar chart can still clearly display one bar.

A line chart can technically represent the point, but there is no real trend yet because a trend requires multiple values.

Zero values

Suppose:

January → 120
February → 0
March → 150

Zero is still valid numerical data.

The chart should show February at the zero level.

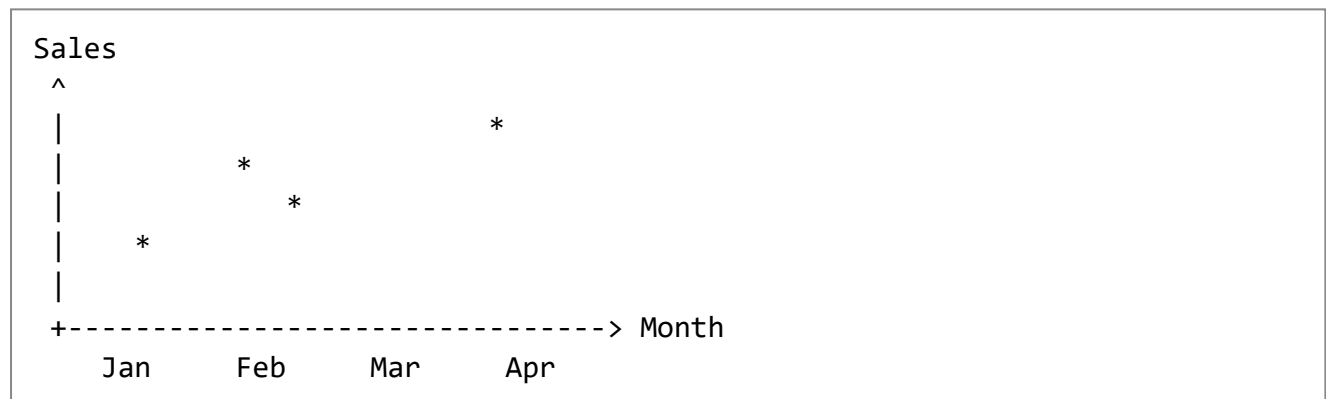
Do not automatically remove a zero unless you know it represents incorrect or missing data.

14. Expected Chart Description

If you use this data:

January → 120
February → 150
March → 135
April → 180

a line chart should approximately communicate:



The chart should have something similar to:

Title: Monthly Sales
X-axis label: Month
Y-axis label: Sales

From the chart you should be able to notice that sales:

- rise from January to February,
- fall slightly in March,
- rise again in April.

That is one major purpose of visualization: **making patterns easier to see.**

15. Hint Only

Start by creating two matching lists:

```
months = [...]  
sales = [...]
```

Then investigate these Matplotlib commands:

```
plt.plot(...)  
plt.title(...)  
plt.xlabel(...)  
plt.ylabel(...)  
plt.show()
```

If you prefer bars instead of a line, investigate:

```
plt.bar(...)
```

Remember the basic relationship:

```
months → x-axis  
sales → y-axis
```

Your goal for Day 78 is simply to turn a tiny numerical dataset into a clear chart with a **title** and **labeled axes**.